

Memoria Técnica — MusicPlayer

1. Introducción

1.1 Descripción del proyecto

MusicPlayer es una plataforma web de reproducción y gestión de música desarrollada como Trabajo de Fin de Grado (TFG). Permite a los usuarios escuchar música, gestionar listas de reproducción, seguir artistas y descubrir nuevo contenido. El sistema soporta tres roles diferenciados: administrador, editor y oyente.

1.2 Objetivos

- Desarrollar una aplicación web completa (frontend + backend) con arquitectura cliente-servidor.
- Implementar un sistema de autenticación basado en tokens JWT con control de acceso por roles.
- Crear una interfaz de usuario moderna e intuitiva inspirada en plataformas de streaming actuales.
- Integrar una API externa para mostrar letras de canciones en tiempo real.
- Documentar el sistema completo para facilitar su mantenimiento y evolución.

1.3 Alcance

El sistema cubre:

- Registro, autenticación y gestión de usuarios.
- Catálogo de artistas, álbumes, géneros y canciones con operaciones CRUD.
- Sistema de listas de reproducción y gestión de canciones favoritas.
- Historial de reproducciones por usuario.
- Dashboards diferenciados según el rol del usuario.
- Obtención de letras de canciones mediante API externa.

2. Arquitectura del sistema

2.1 Visión general

La aplicación sigue una arquitectura de tres capas desacopladas:

```
[Cliente Angular] ↔ [API REST Spring Boot] ↔ [Base de datos MySQL]
                        ↑
                    [API externa: lyrics.ovh]
```

El frontend Angular consume la API REST del backend mediante peticiones HTTP autenticadas con JWT. El backend gestiona la lógica de negocio, la persistencia y la seguridad. Para las letras de canciones, el frontend consulta directamente la API pública `lyrics.ovh`.

2.2 Framework base: JHipster 9.0.0

Se eligió JHipster como generador de código base por los siguientes motivos:

- Genera automáticamente la estructura inicial del proyecto con las mejores prácticas de Spring Boot y Angular.
- Incluye configuración de seguridad JWT preintegrada.
- Proporciona herramientas de migración de base de datos (Liquibase) y mapeo de entidades (MapStruct).
- Reduce el tiempo dedicado a la configuración inicial y permite centrarse en la lógica de negocio y el diseño de interfaz.

2.3 Capa frontend

Construida con **Angular 21** en modo standalone. Se comunica con el backend mediante `HttpClient` y gestiona el estado de sesión a través de JWT almacenado en `localStorage`.

Estructura principal:

src/main/webapp/app/	
└─ app.config.ts	– Configuración de providers (HttpClient, Router, i18n)
└─ app.routes.ts	– Definición de rutas con guards
└─ core/	– Servicios transversales (auth, interceptores, config)
└─ layouts/	– Componentes de maquetación (navbar, sidebar, player-bar)
└─ home/	– Dashboards por rol (admin, editor, usuario)
└─ entities/	– Módulos CRUD de entidades (song, album, artist, etc.)
└─ account/	– Gestión de cuenta de usuario
└─ login/	– Página de autenticación
└─ shared/	– Componentes y utilidades compartidas

2.4 Capa backend

Construida con **Spring Boot 4.0.3** y Java. Expone una API REST protegida por JWT gestionada mediante Spring Security con el módulo OAuth2 Resource Server. Organizada en paquetes con separación clara de responsabilidades:

com.musicplayer	
└─ config/	– 18 clases de configuración (Security, DB, Cache, Mail, etc.)
└─ domain/	– 11 entidades JPA + enumeración AlbumType
└─ repository/	– 12 interfaces Spring Data JPA
└─ service/	– 9 interfaces de servicio + 8 implementaciones
└─ service.dto/	– 11 DTOs de transferencia
└─ service.mapper/	– 10 mappers MapStruct (entidad ↔ DTO)
└─ web.rest/	– 13 controladores REST
└─ web.rest.errors/	– Manejo global de excepciones
└─ security/	– Utilidades y configuración de seguridad JWT

El acceso a cada endpoint está controlado por roles mediante anotaciones Spring Security (`@PreAuthorize`). Las sesiones son completamente **stateless**: cada petición se autentica de forma independiente mediante el token JWT.

2.5 Capa de datos

Base de datos **MySQL 8** en producción; **H2 en memoria** para desarrollo. Las migraciones de esquema se gestionan exclusivamente con **Liquibase** (`spring.jpa.hibernate.ddl-auto: none`), garantizando trazabilidad y reproducibilidad. El esquema incluye 10 changelogs versionados que crean las tablas de negocio, relaciones y roles personalizados de forma incremental.

3. Capa backend

3.1 API REST

El backend expone **13 controladores REST** bajo la ruta base `/api`. Todos los endpoints de negocio requieren autenticación mediante `Authorization: Bearer <token>`.

Recurso	Ruta base	Operaciones
---------	-----------	-------------

Canciones	/api/songs	GET (paginado), GET /{id}, POST, PUT, DELETE
Álbumes	/api/albums	GET (paginado), GET /{id}, POST, PUT, DELETE
Artistas	/api/artists	GET (paginado), GET /{id}, POST, PUT, DELETE
Géneros	/api/genres	GET (paginado), GET /{id}, POST, PUT, DELETE
Playlists	/api/playlists	GET (paginado), GET /{id}, POST, PUT, DELETE
Canciones de playlist	/api/playlist-songs	GET, POST, PUT, DELETE
Reproducciones	/api/plays	GET, POST, DELETE
Favoritos	/api/likes	GET, POST, DELETE
Usuarios (admin)	/api/admin/users	GET, POST, PUT, DELETE
Cuenta	/api/account	GET, POST (actualizar perfil, cambiar contraseña)
Autenticación	/api/authenticate	POST (login), GET (verificar sesión activa)
Registro	/api/register	POST
Activación	/api/activate	GET (?key=...)

Las listas paginadas devuelven la cabecera `X-Total-Count` con el total de registros, compatible con el componente de paginación JHipster del frontend. Las respuestas siguen el estándar HTTP (201 Created en POST, 200 OK en GET/PUT, 204 No Content en DELETE).

3.2 Modelo de dominio

El modelo de datos cuenta con **10 entidades JPA** de negocio:

Entidad	Campos principales	Relaciones
Song	title, duration, fileUrl, coverImage, lyrics, releaseDate	ManyToOne → Album, Genre; ManyToMany ↔ Artist
Album	title, coverImage, releaseDate, albumType	ManyToOne → Artist, Genre
Artist	name, bio, image, country, verified	ManyToMany ↔ Song
Genre	name (único)	—
Playlist	name, description, isPublic, coverImage	ManyToOne → User
PlaylistSong	position, addedAt	ManyToOne → Playlist, Song
Play	playedAt, durationListened	ManyToOne → User, Song
Like	createdAt	ManyToOne → User, Song
User	login, email, firstName, lastName, activated, langKey	ManyToMany ↔ Authority
Authority	name (PK)	—

Todas las entidades de negocio extienden `AbstractAuditingEntity` , que añade automáticamente los campos de auditoría `createdBy` , `createdDate` , `lastModifiedBy` y `lastModifiedDate` mediante Spring Data JPA Auditing.

La enumeración `AlbumType` permite clasificar los álbumes: `ALBUM` , `SINGLE` , `EP` , `PODCAST_SERIES` .

La relación ManyToMany entre `Song` y `Artist` usa una interfaz especializada `SongRepositoryWithBagRelationships` que carga los artistas mediante consultas separadas para evitar el problema de producto cartesiano de Hibernate en colecciones *bag*.

3.3 Capa de servicio y patrón DTO

Se sigue el patrón **Controlador** → **Servicio** → **Repositorio** con DTOs para desacoplar la capa de presentación del modelo de dominio:

1. Los controladores REST reciben y devuelven **DTOs** (11 clases en `service.dto`).
2. Los **servicios** (`service.impl`) contienen la lógica de negocio y orquestan repositorios.
3. Los **mappers MapStruct** (`service.mapper`) convierten automáticamente entre entidades JPA y DTOs generando el código en tiempo de compilación, sin reflexión en tiempo de ejecución.

Flujo de creación de canción como ejemplo:

```
POST /api/songs (SongDTO)
→ SongResource
→ SongService.save(SongDTO)
→ SongMapper.toEntity(SongDTO)
→ SongRepository.save(Song)
→ SongMapper.toDto(Song)
← ResponseEntity<SongDTO> 201 Created
```

Los servicios implementan interfaces (`SongService` , `AlbumService` , `PlaylistService` , etc.) siguiendo el principio de inversión de dependencias, lo que facilita su sustitución y prueba unitaria.

3.4 Persistencia y migraciones Liquibase

Las migraciones están versionadas en `src/main/resources/config/liquibase/`:

Changeset	Contenido
00000000000000_initial_schema.xml	Tablas de usuario JHipster (jhi_user, jhi_authority, jhi_user_authority)
20260410183941_added_entity_Genre.xml	Tabla genre
20260410184041_added_entity_Artist.xml	Tabla artist
20260410184141_added_entity_Album.xml	Tabla album + FK a artist y genre; índices
20260410184241_added_entity_Song.xml	Tabla song + tabla de relación rel_song__artists
20260410184341_added_entity_Playlist.xml	Tabla playlist + FK a usuario
20260410184441_added_entity_PlaylistSong.xml	Tabla playlist_song con campo position
20260410184541_added_entity_Play.xml	Tabla play (historial de reproducciones)

20260410184641_added_entity_Like.xml	Tabla jhi_like (canciones favoritas)
20260501210000_add_editor_artist_authorities.xml	Inserta ROLE_EDITOR y ROLE_ARTIST en jhi_authority

Este enfoque permite desplegar la aplicación en cualquier entorno ejecutando automáticamente solo los changesets pendientes, sin riesgo de duplicar cambios.

3.5 Seguridad del backend

Spring Security gestiona la autenticación y autorización. Las clases principales son `SecurityConfiguration` y `SecurityJwtConfiguration`.

Generación y validación de tokens JWT:

- Biblioteca: **Nimbus JOSE+JWT** (integrada en Spring Security OAuth2 Resource Server)
- Algoritmo: **HS512** (HMAC con SHA-512)
- Validez del token: **1800 segundos** (30 minutos)
- El secreto se configura codificado en Base64 en la propiedad `jhipster.security.authentication.jwt.base64-secret`

Flujo de autenticación:

1. Cliente envía `POST /api/authenticate` con `{username, password, rememberMe}`
2. `AuthenticateController` delega en Spring Security Authentication Manager
3. `DomainUserDetailsService` carga el usuario de la base de datos y verifica credenciales
4. Si válido, `JwtEncoder` genera el token firmado con HS512 → devuelve al cliente
5. El cliente incluye `Authorization: Bearer <token>` en peticiones posteriores
6. `SecurityConfiguration` configura el servidor de recursos OAuth2 que valida el token en cada petición

Control de acceso por endpoint:

```
// Ejemplo: solo ROLE_ADMIN puede crear usuarios
@PreAuthorize("hasAuthority(\"" + AuthoritiesConstants.ADMIN + "\")")
public ResponseEntity<AdminUserDTO> createUser(...) { ... }
```

Roles disponibles (`AuthoritiesConstants.java`):

Constante	Valor	Acceso
ADMIN	ROLE_ADMIN	Gestión completa del sistema y usuarios
USER	ROLE_USER	Oyente: playlists, favoritos, historial
EDITOR	ROLE_EDITOR	Gestión del catálogo musical
ARTIST	ROLE_ARTIST	Gestión del propio contenido

`ROLE_EDITOR` y `ROLE_ARTIST` son roles personalizados añadidos al esquema JHipster base mediante una migración Liquibase dedicada.

3.6 Manejo de errores

El manejo global se centraliza en `ExceptionHandler` (`@ControllerAdvice`), que convierte las excepciones del dominio en respuestas HTTP estructuradas compatibles con **RFC 7807 Problem Details**:

Excepción	HTTP	Descripción
BadRequestAlertException	400	Datos inválidos (incluye nombre de entidad y campo)
LoginAlreadyUsedException	400	Login de usuario ya registrado
EmailAlreadyUsedException	400	Email ya registrado
InvalidPasswordException	400	Contraseña no cumple criterios mínimos
Token inválido / expirado	401	No autenticado
Sin permiso de rol	403	Acceso denegado
Recurso no encontrado	404	Entidad con id solicitado no existe

3.7 Servicio de correo

`MailService` gestiona el envío de emails mediante **Spring Mail** con plantillas **Thymeleaf**:

- **Correo de activación:** se envía al registrar un nuevo usuario; contiene un enlace único a `/api/activate?key=...` que activa la cuenta.
- **Correo de restablecimiento:** se envía al solicitar un reset de contraseña; incluye un token de un solo uso con expiración.

En el perfil de desarrollo la configuración del servidor SMTP apunta a localhost para evitar envíos reales durante el desarrollo y las pruebas.

3.8 Perfiles de despliegue

Perfil	Base de datos	Uso
dev	H2 en memoria	Desarrollo local; esquema recreado en cada arranque
prod	MySQL 8	Producción; datos persistentes entre reinicios
tls	—	Habilita HTTPS/TLS sobre cualquier perfil

Diferencias clave entre perfiles:

- **Dev:** consola H2 accesible en `/h2-console`, logs SQL de Hibernate activos, sin caché L2.
- **Prod:** pool de conexiones HikariCP, logs SQL desactivados, caché de segundo nivel activa, seguridad CORS estricta.

3.9 Monitorización y logging

- **Spring Boot Actuator** expone `/management/health`, `/management/info`, `/management/metrics` y `/management/liquibase`; protegidos para `ROLE_ADMIN`.
 - **AOP Logging:** `LoggingAspect` intercepta todas las llamadas a métodos de los paquetes `repository`, `service` y `web.rest`, registrando entradas, salidas, tiempos y excepciones sin modificar el código de negocio.
 - **Caché L2 de Hibernate:** configurada con Infinispan/EhCache para entidades de referencia frecuentemente leídas (géneros, autoridades).
-

4. Tecnologías del frontend

4.1 Angular 21

Se eligió Angular 21 por:

- **Tipado estático fuerte** con TypeScript, que reduce errores en tiempo de ejecución.
- **Arquitectura basada en componentes standalone**, que simplifica la estructura del proyecto eliminando la necesidad de módulos NgModule.
- **Sistema de señales reactivas** (`signal` , `computed`) para la gestión de estado local sin necesidad de librerías externas.
- **CLI potente** que facilita la generación de código, construcción y pruebas.

4.2 Bootstrap 5 + SCSS

Se empleó Bootstrap 5 para la maquetación responsive y los componentes UI base. Las personalizaciones visuales se realizaron mediante variables SCSS, lo que permite modificar el tema global desde un único punto sin alterar el código de Bootstrap.

4.3 Font Awesome 7

Iconografía vectorial escalable integrada mediante `@fortawesome/angular-fontawesome` . Los iconos se registran globalmente en `app.ts` mediante la librería de iconos, evitando importaciones duplicadas.

4.4 RxJS 7

Utilizado para la gestión de flujos asíncronos (peticiones HTTP, eventos de estado). Se usa principalmente en los servicios de entidades para transformar y manejar respuestas del backend.

4.5 ngx-translate

Internacionalización (i18n) del frontend con soporte para español e inglés. Las traducciones se definen en archivos JSON bajo `src/main/webapp/i18n/` .

5. Diseño UI/UX

5.1 Tema oscuro estilo Spotify

La decisión de adoptar un diseño oscuro similar a Spotify responde a criterios de UX específicos para aplicaciones de música:

- Los fondos oscuros reducen la fatiga visual en sesiones largas de escucha.
- El contraste con portadas de álbumes (generalmente coloridas) destaca el contenido multimedia.
- La familiaridad del patrón de diseño reduce la curva de aprendizaje para el usuario.

Paleta principal:

Token	Color	Uso
<code>\$bg-primary</code>	<code>#0f172a</code>	Fondo general
<code>\$bg-secondary</code>	<code>#1e293b</code>	Cards, sidebar
<code>\$accent</code>	<code>#38bdf8</code>	Elementos activos, hover

\$text-primary	#ffffff	Texto principal
\$text-secondary	#b3b3b3	Texto secundario

5.2 Dashboards diferenciados por rol

Se implementaron tres dashboards distintos que se muestran automáticamente según el rol del usuario autenticado:

Rol	Dashboard	Contenido principal
ROLE_ADMIN	dashboard-admin	Gestión de usuarios, métricas del sistema, accesos directos a entidades
ROLE_EDITOR / ROLE_ARTIST	dashboard-editor	Catálogo musical, gestión de álbumes y canciones propias
ROLE_USER	dashboard-user	Listas de reproducción, canciones favoritas, historial

La redirección al dashboard correcto se realiza en el componente `HomeComponent` mediante comprobación de roles con el servicio `AccountService`.

5.3 Barra lateral (Sidebar)

La sidebar ofrece navegación rápida y varía su contenido según el rol. Incluye:

- Enlace al dashboard de inicio.
- Accesos a entidades relevantes para el rol.
- Estado contraído/expandido gestionado por `SidebarService`.

5.4 Barra del reproductor (Player Bar)

Barra de reproducción fija en la parte inferior de la pantalla, siempre visible. Controles: reproducir/pausar, anterior, siguiente, aleatorio, repetir, volumen y mute. Implementada como componente standalone con señales Angular para el estado interno.

5.5 Panel de letras

Integrado en la barra del reproductor. Al pulsar el botón de letras, se despliega un panel encima de la barra que muestra la letra de la canción en reproducción, obtenida en tiempo real desde la API `lyrics.ovh`. Incluye indicador de carga y mensaje de error si no se encuentran letras.

6. Seguridad del frontend

6.1 Autenticación JWT

Flujo de autenticación:

1. El usuario envía credenciales al endpoint `/api/authenticate`.
2. El backend valida y devuelve un token JWT.
3. El frontend almacena el token y lo incluye en todas las peticiones mediante el interceptor `authInterceptor`.
4. Al expirar el token, `authExpiredInterceptor` redirige automáticamente a la página de login.

6.2 Guards de rutas

Rutas protegidas con `AuthGuard` que verifica la existencia y validez del token antes de permitir la navegación. Las rutas de administración requieren el rol `ROLE_ADMIN`.

6.3 Directivas de autorización por rol

Los elementos de la UI que solo deben mostrarse a ciertos roles se controlan mediante comprobaciones en los componentes con el método `hasAnyAuthority()` del servicio `AccountService`.

7. Integración de la API de letras

7.1 API seleccionada: lyrics.ovh

Se eligió `lyrics.ovh` por:

- **Gratuita y sin clave de API:** no requiere registro ni límites de uso que afecten al TFG.
- **CORS habilitado:** permite peticiones directas desde el navegador sin necesidad de un proxy en el backend.
- **Simple:** endpoint único `GET https://api.lyrics.ovh/v1/{artista}/{titulo}`.

7.2 Implementación

El servicio `LyricsService` encapsula la petición HTTP externa. El componente de la barra del reproductor inyecta este servicio y gestiona los estados de carga, éxito y error mediante señales Angular. El panel se muestra/oculta con un botón en el área derecha de la barra del reproductor.

8. Problemas encontrados y soluciones

8.1 Integración de roles personalizados

JHipster genera por defecto los roles `ROLE_USER` y `ROLE_ADMIN`. Para añadir `ROLE_EDITOR` y `ROLE_ARTIST` fue necesario:

- Añadir los nuevos roles en la tabla `jhi_authority` mediante una migración Liquibase dedicada (`20260501210000_add_editor_artist_authorities.xml`).
- Registrarlos como constantes en `AuthoritiesConstants.java`.
- Adaptar los guards y las comprobaciones de roles en el frontend.

8.2 Diseño responsive de la sidebar y el player

La coexistencia de sidebar fija, navbar superior y player bar inferior requirió un sistema de márgenes y z-index coordinados en el SCSS global para evitar solapamientos en distintos tamaños de pantalla.

8.3 Estado del reproductor sin backend de audio

El reproductor visual está implementado pero la reproducción de audio real depende de que las canciones tengan una URL de archivo válida en el campo `fileUrl`. Para la demo se utilizan estados visuales con señales Angular.

8.4 Carga de relaciones ManyToMany en JPA

La relación `Song ↔ Artist` es ManyToMany. Hibernate genera un producto cartesiano si se usa una única consulta JPQL con `JOIN FETCH`. Se resolvió implementando `SongRepositoryWithBagRelationships`, que separa la carga en dos consultas distintas: una para las canciones y otra para sus artistas, evitando duplicados y mejorando el rendimiento.

9. Conclusiones y trabajo futuro

9.1 Conclusiones

El proyecto ha permitido aplicar en un caso real los conocimientos adquiridos durante el grado: arquitectura cliente-servidor, patrones de diseño de interfaces, seguridad basada en tokens y consumo de APIs externas. La elección de JHipster como base generó una estructura sólida que permitió centrarse en la personalización del frontend y la lógica de negocio desde el primer día. Angular 21 con señales y componentes standalone resultó en un código más conciso y predecible comparado con versiones anteriores del framework.

9.2 Trabajo futuro

- Reproducción de audio real integrando un servicio de almacenamiento de ficheros (S3, Cloudinary).
- Letras sincronizadas con la reproducción (formato LRC) mediante `lrc1ib.net`.
- Sistema de recomendación de canciones basado en el historial de reproducciones.
- Aplicación móvil nativa con el mismo backend (Ionic o React Native).
- Despliegue en producción con CI/CD automatizado (GitHub Actions + Docker).
- Implementación de tests de integración con Testcontainers para el backend.